Machine Learning
Fall 2026

Faculty of Computers and AI
Cairo University

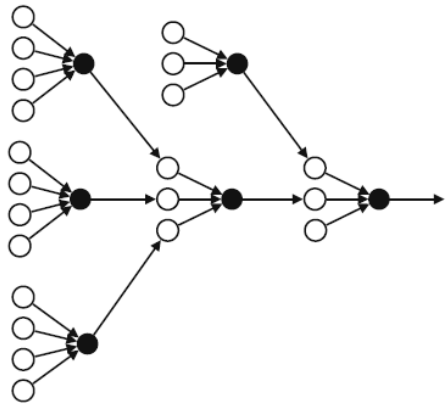
Lab7: Neural Networks (NNs)

Overview:

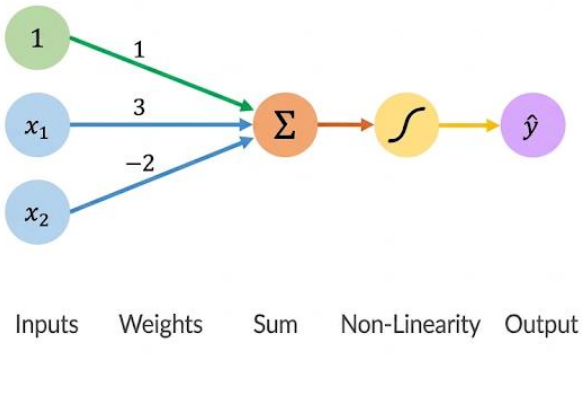
Neural networks, also called multi-layer perceptrons. Neural networks are one of the most popular machine learning models out there. They are so useful that the field has its own name: ***deep learning***. Neural networks are inspired by how the human brain operates. Neural networks can be seen as composition of linear classifiers that give rise to nonlinear classifiers. They are useful for classification and regression. Moreover, neural networks can be used to extract patterns from non-linearly separable data in order to classify them.

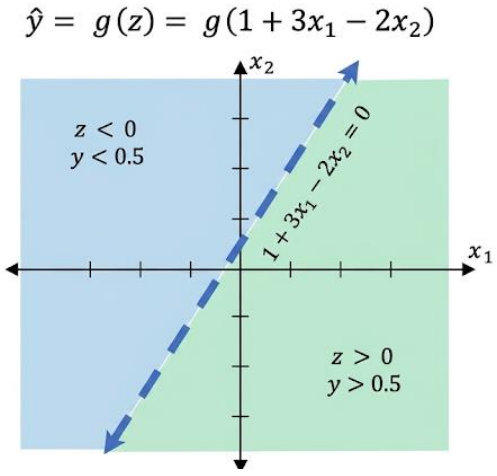
A single neuron has the following entities:

Input: $x_1, x_2, \dots, x_n$
Weights: $w_1, w_2, \dots, w_n$
Bias: b
Activation function: i.e. sigmoid function.
Prediction: $\hat{y} = f(w_1x_1 + w_2x_2 + \cdots w_nx_n)$ where f is the corresponding activation function.



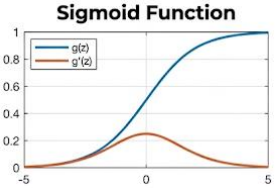
Logistic regression: is one of the machine learning algorithms that can be used to classify linearly separable data by drawing linear decision boundaries between them.





Activation functions: can help in introducing non-linearity to linear transformations to extract new features through the network's layers.

Sigmoid Function

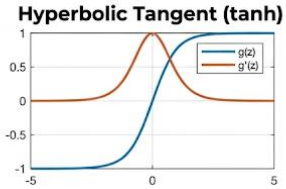


$$g(z) = \frac{1}{1+e^{-z}}$$

$$g'(z) = g(z)(1-g(z))$$

`tf.math.sigmoid(z)`

Hyperbolic Tangent (tanh)

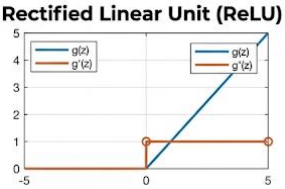


$$g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

$$g'(z) = 1-g(z)^2$$

`tf.math.tanh(z)`

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

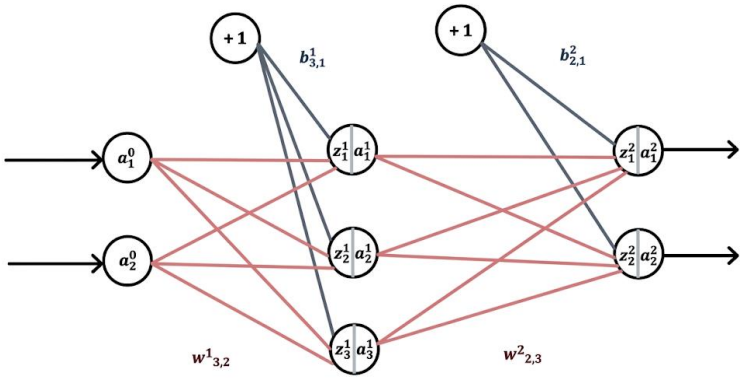
$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

`tf.nn.relu(z)`

Training Neural Networks:

We begin by initializing all the weights and biases at random. Next, we define an error function to measure the performance of the neural network. Finally, we repeatedly use the error function to tune in the weights and biases of the model to reduce the error function.

Steps of NNs Training:



Input layer Hidden layer Output layer

(1) Forward pass: Inputs flow through the network, computing linear combinations, passing through the nonlinear activation function and producing an output prediction.

$$a^0 = input$$

$$z^1 = w^1 a^0 + b^1$$

$$a^1 = f_1(z^1)$$

$$z^2 = w^2 a^1 + b^2$$

$$a^2 = f_2(z^2)$$

(2) Error function: The loss function measures the difference between prediction and true labels.

$$c = \frac{1}{2}\sum_i (a_i^2 - y_i)^2$$

(3) Backpropagation:

- Calculate the gradient of the loss function (namely, the derivatives with respect to each one of the weights and biases) using chain rule.
- Update weights by taking a small step in the direction opposite to the gradient to decrease the loss function by a small amount.

$$w^2_{new} = w^2 - \alpha\left(\frac{\partial c}{\partial w^2}\right) = w^2 - \alpha\left(\frac{\partial c}{\partial a^2}\frac{\partial a^2}{\partial z^2}\frac{\partial z^2}{\partial w^2}\right)$$

$$b^2_{new} = b^2 - \alpha\left(\frac{\partial c}{\partial b^2}\right) = b^2 - \alpha\left(\frac{\partial c}{\partial a^2}\frac{\partial a^2}{\partial z^2}\frac{\partial z^2}{\partial b^2}\right)$$

$$w^1_{new} = w^1 - \alpha\left(\frac{\partial c}{\partial w^1}\right) = w^1 - \alpha\left(\frac{\partial c}{\partial a^2}\frac{\partial a^2}{\partial z^2}\frac{\partial z^2}{\partial a^1}\frac{\partial a^1}{\partial z^1}\frac{\partial z^1}{\partial w^1}\right)$$

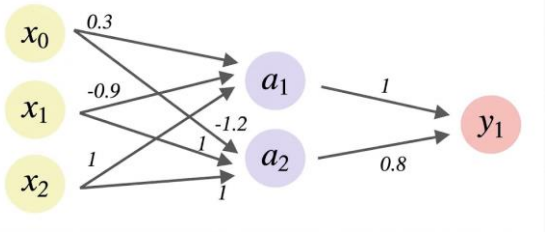
$$b^1_{new} = b^1 - \alpha\left(\frac{\partial c}{\partial b^1}\right) = b^1 - \alpha\left(\frac{\partial c}{\partial a^2}\frac{\partial a^2}{\partial z^2}\frac{\partial z^2}{\partial a^1}\frac{\partial a^1}{\partial z^1}\frac{\partial z^1}{\partial b^1}\right)$$

$$w^2 ::= w^2_{new}, \qquad b^2 ::= b^2_{new}$$
$$w^1 ::= w^1_{new}, \qquad b^1 ::= b^1_{new}$$

Example: We want to train the FFNN below to perform XNOR. How will the training be performed? Perform backpropagation on 1 training example. *(Use sigmoid activation function & learning rate = 0.3)*

Training data (XNOR truth table):

x_1	x_2	y_1
0	0	1
0	1	0
1	0	0
1	1	1



Now, we will perform backpropagation for 1 training example. Let's take the second one where $x_1 = 0, x_2 = 1, y_1 = 0$ and x_0 is always 1. So, we have:

$$x = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

$$w^h = \begin{bmatrix} 0.3 & -0.9 & 1 \\ -1.2 & 1 & 1 \end{bmatrix}$$

$$w^o = [1 \quad 0.8]$$

Feed forward step:

$$a^h = \begin{bmatrix} a_1^h \\ a_2^h \end{bmatrix} = f(w^h x) = f\left(\begin{bmatrix} 0.3 & -0.9 & 1 \\ -1.2 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} f(1.3) \\ f(-0.2) \end{bmatrix} = \begin{bmatrix} \frac{1}{1+e^{-1.3}} \\ \frac{1}{1+e^{0.2}} \end{bmatrix} = \begin{bmatrix} 0.786 \\ 0.45 \end{bmatrix}$$

$$a^o = a_1^o = f(w^o a^h) = f\left(\begin{bmatrix} 1 & 0.8 \end{bmatrix} \begin{bmatrix} 0.786 \\ 0.45 \end{bmatrix} \right) = f(1.146) = \frac{1}{1+e^{-1.146}} = 0.759$$

Backpropagation step:

$$w_{ji}^h = w_{ji}^h - \eta \ \delta_j^h \ a_i^{h-1}$$

$$\boldsymbol{\delta_j^l} = \begin{cases} \delta_k^o = (a_k^o - y_k) \ . \ \dfrac{\partial f(\text{net}_k^o)}{\partial \text{net}_k^o} & \text{If } l \text{ is the output layer } o \\ \left(\sum_{k=1}^{\text{len}(l+1)} \delta_k^{l+1} . \ w_{kj}^{l+1} \right) . \ \dfrac{\partial f(\text{net}_j^l)}{\partial \text{net}_j^l} & \text{Otherwise} \end{cases}$$

Error terms for output layer:

$$\boldsymbol{\delta_1^o} = (a_1^o - y_1) \, . \, a_1^o \, . \, (1 - a_1^o) = (0.759 - 0) \times 0.759 \times (1 - 0.759) = 0.139$$

Error terms for hidden layer:

$$\boldsymbol{\delta_1^h} = \left(\sum_{k=1}^{1} \delta_k^o . \ w_{k1}^o \right) . \, a_1^h \, . \, (1 - a_1^h)$$

$$= \delta_1^o \times w_{11}^o \times 0.786 \times (1 - 0.786) = 0.139 \times 1 \times 0.786 \times (1 - 0.786) = 0.023$$

$$\boldsymbol{\delta_2^h} = \left(\sum_{k=1}^{1} \delta_k^o . \ w_{k2}^o \right) . \, a_2^h \, . \, (1 - a_2^h)$$

$$= \delta_1^o \times w_{12}^o \times 0.45 \times (1 - 0.45) = 0.139 \times 0.8 \times 0.45 \times (1 - 0.45) = 0.028$$

Weight update (output layer):

$$w_{11}^o = w_{11}^o - \eta \ \delta_1^o \ a_1^h = 1 - (0.3 \times 0.139 \times 0.786) = 0.967$$

$$w_{12}^o = w_{12}^o - \eta \ \delta_1^o \ a_2^h = 1 - (0.3 \times 0.139 \times 0.45) = 0.78$$

Weight update (hidden layer layer):

$$w_{10}^h = w_{10}^h - \eta \ \delta_1^h \ x_0 = 0.3 - (0.3 \times 0.023 \times 1) = 0.29$$
$$w_{11}^h = w_{11}^h - \eta \ \delta_1^h \ x_1 = -0.9 - (0.3 \times 0.023 \times 0) = -0.9$$

$$w_{12}^h = w_{12}^h - \eta\ \delta_1^h\ x_2 = 1 - (0.3 \times 0.023\ \times 1) = 0.99$$

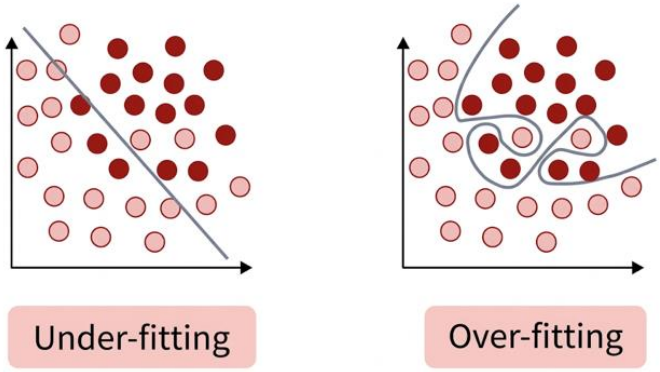
$$w_{20}^h = w_{20}^h - \eta\ \delta_2^h\ x_0 = -1.2\ - (0.3\ \times 0.028\ \times 1) = -1.2$$
$$w_{21}^h = w_{21}^h - \eta\ \delta_2^h\ x_1 = \ 1 - (0.3\ \times 0.028\ \times 0) = 1$$
$$w_{22}^h = w_{22}^h - \eta\ \delta_2^h\ x_2 = 1 - (0.3\ \times 0.028\ \times 1) = 0.99$$

Pseudocode for the backpropagation algorithm:

- Initialize the neural network with random weights and biases.
- Repeat many times:
  - Perform forward propagation
  - Calculate the loss function and its gradient (namely, the derivatives with respect to each one of the weights and biases).
  - Update weights by taking a small step in the direction opposite to the gradient to decrease the loss function by a small amount.
- The weights you obtain correspond to a neural network that (likely) fits the data well.

Potential problems in NNs:

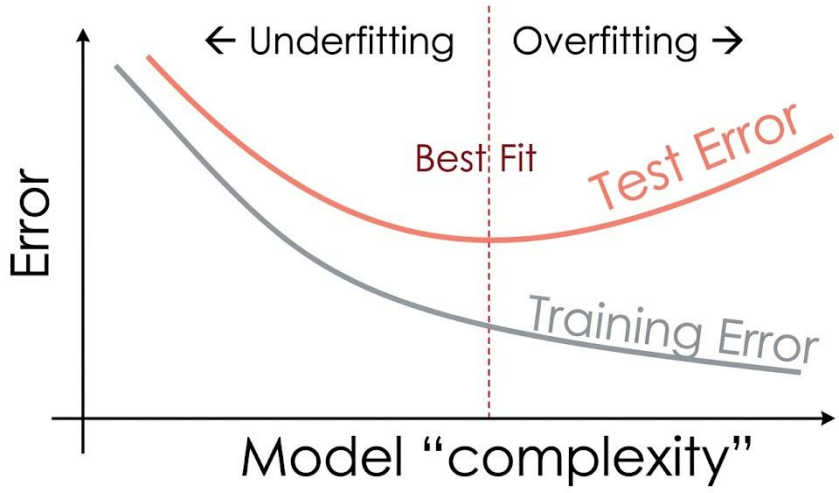
In practice, neural networks work very well. But, due to their complexity, many problems arise with their training.



Overfitting: **using a very complex model to fit our data leading to** large network architectures that can potentially memorize our data without generalizing it well to new data. We can apply techniques to reduce the amount of overfitting that your network may have:

- **Regularization:** add regularization term to error function to penalize higher weights.
- **Dropout:** Making sure a few strong nodes are not dominating the training.

Underfitting: **using a simple model for complex data**. It happens when we try to train a model that is too simple, and it is unable to learn and capture the complexities of the data.



References:

[1] Serrano, L. (2021). *Grokking Machine Learning*. Manning Publications.